Architettura degli Elaboratori a.a. 2024/25

Appello d'esame 11/09/2025 (ASM)

Operazioni preliminari

1. Aprire il file `AE - Uso del computer in laboratorio per la prova ASM.pdf`, contenente informazioni utili per lo svolgimento di questa prova.
2. Aprire tramite RARS il file `program01.asm` in questa directory
3. Completare le seguenti righe con i propri dati in `program01.asm`

```
########################################
# INSERIRE I PROPRI DATI QUI:
# Nome:
# Cognome:
# Matricola:
########################################
```

Esercizio

Sia data in input da utente una stringa di caratteri `S_in`. La stringa è lunga al più 50 caratteri (non è necessario verificare questo vincolo) e consiste di un numero intero in notazione esadecimale: '`0`' (codice ASCII `0x30`), '`1`' (codice ASCII `0x31`), ..., '`9`' (codice ASCII `0x39`), '`a`' (codice ASCII `0x61`), '`b`' (codice ASCII `0x62`), ..., '`f`' (codice ASCII `0x66`) il numero termina con il carattere accapo (codice ASCII `0x0a`). Si realizzi, in assembly RISC-V, un programma che somma le cifre che compongono il numero in ingresso. Tale procedura deve essere reiterata (usando il numero di output come nuovo input) finché il numero ottenuto non si riduce ad una singola cifra. Si ricorda che ogni carattere in forma testuale è rappresentato con 8 bit, mentre una cifra di un numero esadecimale (ad esempio quando letto da un registro) è rappresentato con 4 bit. La stringa di output deve essere formata dal carattere '`>`' (codice ASCII `0x3e`) seguito dal valore ottenuto sommando le cifre dell'input ad ogni step; la stringa deve terminare con un carattere di accapo `0x0a`. I numeri visualizzati a schermo devono essere formati sempre da otto cifre esadecimali (che è la rappresentazione di default usando la ecall di stampa in esadecimale).

Se, ad esempio, `S_in` fosse `9ae5` allora dovrebbe essere scritto a schermo (il carattere di accapo finale non è mostrato):

```
>0x00000026>0x00000008
```

perché 0x9+0xa+0xe+0x5 equivale a 9+10+14+5=38, ovvero 0x26; prendendo questo numero come nuovo input, si ottiene 2+6=8 ovvero 0x8.

Nella sezione dedicata al **text segment**, si consiglia la seguente struttura: i) il caricamento dei dati da input (già fornito nel file `program01.asm`: l'indirizzo base della stringa `S_in` verrà caricato nel registro `a0`, ii) il codice necessario a calcolare i valori e visualizzare i risultati sullo schermo, iii) la chiamata ad una funzione **`sommaCifre`**, definita di seguito, e iv) la stampa a terminale del risultato. Si consiglia che la stampa a terminale non avvenga all'interno della funzione `sommaCifre`, ma nel comparto *main* chiamante o in una funzione dedicata di propria stesura. Fare attenzione al fatto che a0 (indirizzo base di `S_in`) possono essere sovrascritto da chiamate ad altre funzioni o ecall.

La funzione `sommaCifre` può ad esempio accettare come argomento:

- `a0`: il numero di cui sommare le cifre;

- `a1`: il nuovo numero ottenuto dalla somma delle cifre.

Note: Commentare le righe più importanti del codice avendo cura di spiegare a cosa servano i registri. Si ricorda che la syscall per la stampa di un carattere prevede come argomenti `a7=11` e `a0` assegnato con il carattere da stampare. La syscall per la stampa di un numero in rappresentazione esadecimale prevede come argomenti `a7=34` e `a0` assegnato con il numero da stampare.

Risultato atteso

Per ogni input, al programma va stampato l'output della procedura suddetta seguito da accapo come spiegato precedentemente.

Ad esempio, il file `test-03.in` contiene il seguente input: '`123456789abcdef`', il suo output atteso (all'interno di `test-03.expt`) è:

```
>0x00000078>0x0000000f
```

È possibile studiare i casi di test aprendo i file di input (nel formato `test-`*xy*`.in`, dove *xy* è un numero a due cifre) e output atteso (estensione `test-`*xy*`.expt`), confrontandoli col proprio (estensione `test-`*xy*`.out`).

Verifica di corretta esecuzione dell'esercizio

Per verificare che l'esercizio sia stato completato correttamente eseguire `run.sh` (doppio click sul file dal file manager, oppure esecuzione del comando `./run.sh` da terminale) e visualizzare il risultato aprendo il file `test_results.html`. Per ulteriori informazioni, consultare il file `AE - Uso del computer in laboratorio per la prova ASM.pdf`.

Bonus

Una soluzione pienamente funzionante, dove ogni test verrà completato usando meno di 350 istruzioni sarà premiata con un bonus di **1 punto** sul voto finale.

Note

- Non è consentito modificare il *data segment*.
- Il limite massimo di istruzioni eseguibili è `305900`. Oltre quel numero, l'esecuzione viene automaticamente terminata.
- Il file `program01.vuoto.asm` contiene una copia del file `program01.asm` che può essere utile in caso sia necessario ripartire da capo.
- Attenzione a non eseguire loop infiniti con leak della memoria: RARS potrebbe crashare e cancellare il file che state scrivendo, sentitevi liberi di salvare un file di backup prima di eseguire del codice rischioso.